UK TRAIN RIDES

Final Project Deliverables Guide

A detailed, practical checklist for taking the project from raw data to business insights and forecasts.

01 Understand the data	**02** Clean and preprocess
03 Build the SQL data model	**04** Analyze business questions
05 Forecast future demand	**06** Build a Tableau dashboard
07 Report and present	**Final** Package all deliverables

Project objective: turn train journey and ticket data into reliable analysis, useful forecasts, and an interactive dashboard that supports business decisions.

Use this document as the team’s master checklist. Some tasks depend on the actual dataset, so verify that each question and feature is supported by available columns before implementing it.

Part 1 — Understand the Data

Goal: learn what the dataset contains before making changes.

Tasks

- Load the source file with pandas, for example: `df = pd.read_csv('file.csv')`.
- Inspect the dataset size with `df.shape`, preview rows with `df.head()`, inspect column types and non-null counts with `df.info()`, and review numeric summaries with `df.describe()`.
- Create a data dictionary that explains every column, its meaning, its data type, and example values.
- Identify key fields needed for analysis: journey or ticket ID, date/time, departure and arrival stations, route, ticket type/class, price, payment method, journey status, and delay—only where those fields exist.
- Check the source, date range, row granularity, currency, and whether one row represents a ticket, booking, or completed ride.

Expected output

- Dataset profile and data dictionary.
- A short note describing what one row represents, the time period covered, and known data limitations.

Why this matters: you cannot safely clean, analyze, or forecast data until you understand what each row and column means.

Part 2 — Clean and Preprocess the Data

Goal: make the dataset consistent and suitable for analysis.

Tasks

- **Missing values:** count missing values with `df.isna().sum()`. Decide column by column whether to fill values (for example, median for appropriate numeric fields, most-common category, or 'Unknown') or exclude records. Document the reason.
- **Duplicates:** identify repeated records, check whether they are truly duplicates, then remove or resolve them.
- **Data types:** convert dates and times to appropriate datetime types and prices/delays to numeric types. Record conversion failures.
- **Inconsistent text:** trim spaces, standardize capitalization, and map spelling variants to consistent category names.
- **Invalid values and outliers:** investigate negative prices, impossible durations, implausible delays, and extreme values. Do not delete an outlier solely because it is large; confirm whether it is an error.
- **Feature engineering:** where source fields allow it, create day of week, month, hour, weekend flag, journey duration, delay in minutes, and revenue per ride/ticket.
- **Validation:** compare row counts and summary statistics before and after cleaning; check that transformations did not silently remove valid records.
- **Save the result:** export a clean CSV or another agreed format and keep the original raw file unchanged.

Expected output

- Cleaned dataset ready for analysis.
- Preprocessing notebook with code, explanations, cleaning decisions, and validation checks.
- A brief before-versus-after quality summary.

Important: calculate revenue using a clearly defined business rule. Ticket price, recognized revenue, refunds, and cancellations may not mean the same thing.

Part 3 — Build the SQL Data Model

Goal: organize the data into tables that can be queried reliably.

Suggested design: a star schema

A star schema uses a central fact table for measurable events and dimension tables for descriptive information. Adjust the design to match the actual data and the intended row granularity.

Table	Typical contents	Purpose
FactTrainRide	Ticket/ride ID, date key, route key, class key, price, delay, status	Stores one defined ride or ticket event and its measurable values.
DimDate	Date key, date, day, month, quarter, year, weekday/weekend	Supports time-based analysis.
DimRoute / DimStation	Route key, departure station, arrival station; or station attributes	Supports route and station comparisons.
DimTicketClass	Class key, class label	Compares demand and revenue by class.
DimTicketType	Ticket type key, ticket type	Compares ticket categories.
DimPaymentMethod	Payment method key, method label	Analyzes payment-method patterns.

Tasks

- Define the grain of the fact table (exactly what one row represents).
- Choose primary keys for uniqueness and foreign keys to connect fact and dimension tables.
- Write SQL scripts to create tables and load the cleaned data.
- Run JOIN queries and validation checks to confirm that relationships work and row counts are sensible.
- Draw an entity-relationship or schema diagram and include it in the report.

Expected output

- SQL table-creation and loading scripts.
- Test queries and data-model diagram.

Modeling caution: if the dataset has separate bookings, tickets, and actual journeys, do not treat them as interchangeable. Avoid double-counting revenue or rides.

Part 4 — Business Analysis: What Happened?

Goal: answer decision-makers' questions with evidence.

1. Create question groups

- **Revenue:** What is total revenue? How does revenue vary by route, month, ticket class, or ticket type?
- **Demand:** Which routes and stations are busiest? Which days, months, or hours show the highest volume?

- **Customers and purchases:** Which ticket types and payment methods are common? Are advance purchases different from same-day purchases, if purchase-time data exists?
- **Operations:** How do delays and cancellations vary? Are they associated with refunds or revenue changes, if those fields exist?

2. Answer and communicate each question

- Confirm that the dataset contains the columns needed to answer each question; remove or revise unsupported questions.
- Use SQL and/or pandas to calculate the required metrics.
- Choose a suitable chart: bar charts for category comparisons, line charts for trends, and heatmaps for patterns across days and hours.
- Write a concise insight for each result, including the metric and relevant time period. Avoid unsupported claims of causation.

Expected output

- A documented list of answerable analysis questions.
- SQL queries and/or analysis notebook cells, charts, and written findings.

Example insight format: “Route A had the highest recorded ticket revenue in the selected period.” Add the measured value and period when available.

Part 5 — Forecasting: What Will Happen?

Goal: create and evaluate forecasts for rides, daily revenue, and demand by ticket class.

Required forecasting questions

- How many rides are expected next month?
- What revenue is expected on each day of next month?
- What demand is expected for each ticket class?

Workflow

- Aggregate the historical data into a daily time series: one row per date with ride count, revenue, and ride counts by ticket class.
- Check date coverage and missing dates. Distinguish a true zero-demand day from a missing or incomplete data day.
- Create features where supported: weekday, month, weekend flag, holiday flag, time trend, and lag features such as the previous day and previous week. Ensure features use only information available at prediction time.
- Split data chronologically: train on older observations and test on the most recent period. Never randomly shuffle time-series observations for this evaluation.
- Establish a simple baseline, such as using the value from the same weekday in the previous week.
- Train candidate models using scikit-learn. Start with a baseline and a simple regression model, then compare suitable tree-based models such as Random Forest or Gradient Boosting.
- Evaluate models using MAE and RMSE; use R^2 where it is meaningful. Compare results against the baseline and inspect errors across time.
- Choose a model based on held-out performance and the forecasting setup. Generate a daily forecast for the entire next month.

- Sum daily ride predictions for the monthly total. Forecast daily revenue directly or estimate it from predicted rides and a defensible average-price model.
- Forecast demand separately by ticket class, or use a multi-output approach if appropriate. Check that predicted class totals are coherent with total rides.
- Plot history and forecast together, clearly marking the forecast period. Add uncertainty intervals if your method supports them.

Expected output

- Reproducible forecasting notebook and saved model/configuration details.
- Model comparison and evaluation results.
- Forecast charts for monthly rides, daily revenue, and ticket-class demand.

Forecasting caution: future data is unknown. Do not use future values to build lag features or tune the model. If prices, refunds, holidays, or service changes matter, state whether they are modeled or omitted.

Part 6 — Build the Tableau Dashboard

Goal: make findings easy for managers to explore.

Tasks

- Connect Tableau to the cleaned dataset or SQL tables and verify field types and relationships.
- Create visualizations for the key analysis questions and all three required forecasting questions.
- Add practical filters such as date range, route/station, ticket class, and ticket type where those fields exist.
- Arrange charts into a clear dashboard with a descriptive title, consistent formatting, and readable labels.
- Place forecast visuals near the relevant historical charts and clearly distinguish actual values from predictions.
- Check that filters work, values reconcile with SQL/Python outputs, and the dashboard is readable at presentation size.

Expected output

- Tableau workbook or published dashboard, depending on the submission requirements.
- Dashboard screenshots for the final report.

Part 7 — Final Report and Presentation

Goal: explain the problem, methods, results, and business value.

Recommended report structure

- Introduction and problem statement.
- Dataset description, data dictionary, coverage, and limitations.
- Data cleaning and preprocessing decisions.
- SQL data model and schema diagram.
- Business analysis questions, charts, answers, and key findings.
- Forecasting approach, model comparison, evaluation metrics, forecast results, and caveats.
- Tableau dashboard screenshots and explanation.
- Conclusions and actionable recommendations for the train company.
- Limitations and future improvements.

Presentation

- Use a short slide deck that tells the same story as the report.
- Lead with the business problem and the most important findings.
- Show selected charts, forecasting results, and the dashboard.
- Finish with recommendations, limitations, and possible next steps.

Expected output

- Final written report.
- Presentation slide deck.

Final Deliverables Checklist

Use this list to verify that the complete project is ready to submit.

Area	Deliverable	Done
Data understanding	Dataset profile and data dictionary	■
Data preparation	Cleaned dataset	■
Data preparation	Preprocessing notebook with documented decisions and validation	■
SQL data model	SQL scripts to create and load tables	■
SQL data model	Schema / entity-relationship diagram	■
Business analysis	Analysis questions with SQL/Python answers and visualizations	■
Forecasting	Model comparison and evaluation metrics	■
Forecasting	Forecast plots: rides, daily revenue, and ticket-class demand	■
Dashboard	Tableau dashboard/workbook	■
Reporting	Final report with methods, results, recommendations, and limitations	■
Presentation	Final presentation slides	■

Suggested project folder structure

```
uk-train-rides/
■■■ data/
■ ■■■ raw/   (original files; keep unchanged)
■ ■■■ processed/   (cleaned data)
■■■ notebooks/   (preprocessing, analysis, forecasting)
■■■ sql/   (schema, loading, validation, analysis queries)
■■■ models/   (saved model artifacts, if applicable)
■■■ visuals/   (exported charts and schema diagram)
■■■ tableau/   (workbook and dashboard exports)
■■■ report/   (final report and presentation)
■■■ README.md   (setup steps, data source, and how to reproduce results)
```

Definition of done: another person should be able to understand the dataset, reproduce the cleaning and analysis, inspect the SQL model, evaluate the forecasts, explore the dashboard, and follow the conclusions in the report.